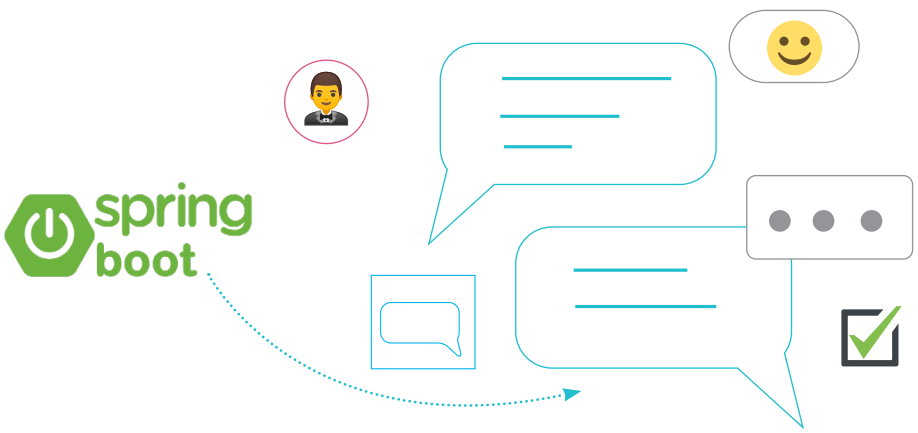
Building a STOMP Messaging Application with Spring Boot

Here's a brief guide on how to build a STOMP-based messaging application using Spring Boot.



In modern web applications, real-time communication is a critical feature, particularly in applications such as stock market tracking platforms and ticket booking systems. Users expect instantaneous updates without the need to refresh their browsers. WebSockets, combined with STOMP (Simple Text-Oriented Messaging Protocol), provide an effective solution for handling bi-directional, full-duplex communication in such scenarios. We will now explore how to build a STOMP-based messaging application using Spring Boot, leveraging open source frameworks for seamless integration.

Solution architecture

The architecture for a STOMP messaging application consists of the following key components.

Spring Boot web application: A backend service handling WebSocket connections and broadcasting messages.

STOMP communication: A messaging protocol that enables WebSocket-based communication between clients and servers.

Message Broker (RabbitMQ/ActiveMQ or Spring's Simple Broker): Facilitates message routing between connected clients.

WebSocket clients: Frontend clients subscribing to real-time updates from the server.

In this architecture, WebSocket clients connect to a Spring Boot application that acts as a WebSocket server. This server, in turn, interacts with a message broker to distribute messages among connected clients efficiently.

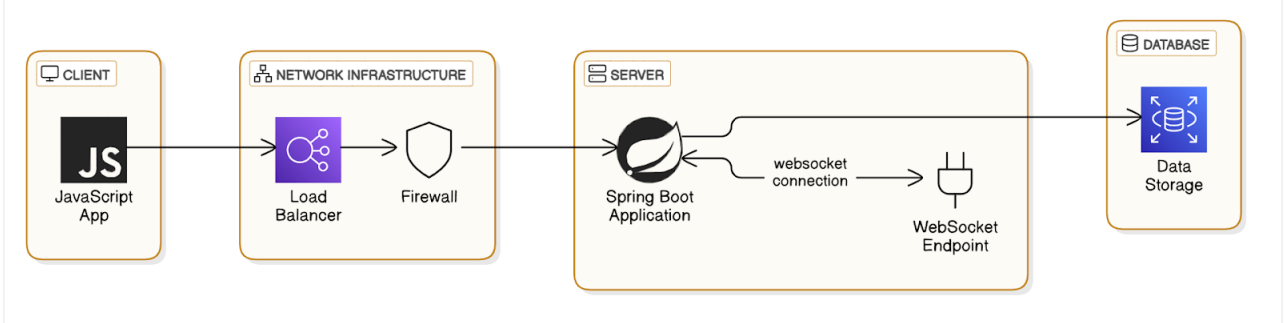


Figure 1: Technical architecture of WebSocket-based interactive web application